

SUMMER TRAINING REPORT
on
AI-BASED TRAFFIC SURVEILLANCE AND ANALYTICS

Training Details

School of Artificial Intelligence and Data Science (SAIDE)
NH-65 Nagaur Road, Karwar, Jodhpur 342030, INDIA

Indian Institute of Technology Jodhpur

Training INCHARGE

Dr. Divya Saxena
Assistant Professor-I

PERIOD

01 June 2026 to 20 July 2026

SUBMITTED BY

Samridhi Kulshrestha (2401030182)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY,
NOIDA (U.P.) SEPTEMBER 2026



School of Artificial Intelligence and Data Science (SAIDE)

Indian Institute of Technology Jodhpur
NH-65 Nagaur Road Karwar, Jodhpur 342030, INDIA
eMail: divyasaxena@iitj.ac.in

Divya Saxena
Assistant Professor - I
30 August 2026

To,

Ms. Samridhi Kulshrestha B.Tech. 2nd Year
Jaypee Institute of Information Technology (JIIT),
Sector-62, Noida
Noida, India

Subject: Completion Letter for Summer Internship 2026

Dear Ms. Kulshrestha,

It gives me great pleasure to formally certify that **Samridhi Kulshrestha, Enrollment No. 2401030182, student of Jaypee Institute of Information Technology (JIIT), Sector-62, Noida** has done internship under my supervision **at the School of Artificial Intelligence and Data Science, Indian Institute of Technology Jodhpur, from 01 June 2026 to 20 July 2026** on the topic **“AI-Based Traffic Surveillance and Analytics.”**

During the training period, she worked with dedication and sincerity, demonstrating a consistent approach towards the assigned work. She showed good interest in understanding the concepts and completed the assigned tasks systematically. Her conduct, discipline, and overall performance throughout the training were found to be satisfactory.

I appreciate her efforts and wish her every success in her future academic and professional endeavors.

Divya Saxena

Dr. Divya Saxena

DECLARATION

I, Samridhi Kulshrestha, hereby declare that this summer training report entitled “IntelliTraffic AI: An AI-Powered Traffic Surveillance, Speed Estimation and License Plate Recognition System”, submitted to Jaypee Institute of Information Technology, Noida, in partial fulfilment of the requirements for the degree of Bachelor of Technology, is a bona fide record of the work done by me during the course of my summer training. The contents and facts presented in this report have been prepared by me without any bias and are authentic to the best of my knowledge.

I also declare that this work has not previously formed the basis for the award of any degree, diploma, fellowship, or other similar title of any institute or society.

Name: Samridhi Kulshrestha

Enrollment: 2401030182

Date: 05/08/26

ACKNOWLEDGEMENT

The time I spent on this summer training, extending and rebuilding my existing IntelliTraffic project from 01 June to 20 July, 2026, has been a rewarding experience. Working through the detection, tracking, speed-estimation, and license-plate-recognition pipeline end to end gave me a much deeper appreciation of how individual computer-vision components come together into a working system, and helped me discover my own potential as a developer and researcher.

With a deep sense of gratitude, I acknowledge everyone who directly or indirectly contributed to this work. I express my sincere thanks to my Training Incharge, **Dr. Divya Saxena**, for the technical guidance and constant supervision that contributed immensely to the successful completion of this project, and for the encouragement that helped me push the project from a working prototype toward a properly tested and documented system.

I would also like to thank my parents and my friends, whose love, patience, and support accompany me in everything I pursue. Nobody has been more important to me than my family in this journey.

Finally, I express my sincere gratitude to the Training and Placement Cell, Jaypee Institute Of Information Technology, for their continuous guidance, encouragement, and support throughout the training period. I am also grateful for the opportunity to undertake this training at JIIT, which provided me with a valuable environment to develop my technical skills and gain practical experience in computer vision and intelligent transportation systems.

Samridhi Kulshrestha

TABLE OF CONTENTS

CHAPTER 1	9
1.1 Background and Motivation	9
1.2 Objectives of the Project	9
1.3 Scope of the Report	10
CHAPTER 2	11
2.1 Object Detection with YOLO	11
2.2 Multi-Object Tracking	11
2.3 Vision-Based Vehicle Speed Estimation	11
2.4 Automatic License Plate Recognition (ALPR)	12
2.5 Traffic Analytics Dashboards	12
2.6 Summary of the Literature Survey	13
CHAPTER 3	14
3.1 Overall System Architecture	14
3.1.1 Configuration-Driven Design	14
3.2 Vehicle Detection and Tracking Algorithm	15
3.3 Speed Estimation Algorithm	16
3.4 License Plate Recognition Algorithm	17
3.5 Overspeed Violation Detection Logic	18
CHAPTER 4	20
4.1 Technology Stack	20
4.2 Project Structure	20
4.3 Key Implementation Choices	21
4.3.1 Strict Vehicle-Class Filtering	21
4.3.2 Browser-Playable Video Output	21
4.3.3 Graceful Failure Handling	21
4.4 Testing and Quality Assurance	21
4.5 Research and Evaluation Toolkit	22
CHAPTER 5	23
5.1 Automated Test Results	23
5.2 Dashboard Walkthrough	23
5.2.1 Live Processing	23
5.2.2 Overview	24
5.2.3 Violations	24
5.2.4 Search	25

5.2.5 Trajectories.....	25
5.3 Sample Output Artefacts.....	26
5.4 Deployment.....	27
5.5 Discussion.....	28
CHAPTER 6.....	29
6.1 Summary of Work Carried Out	29
6.2 Learning Outcomes.....	29
6.3 Relevance to Current Industry Practice	30
6.4 Limitations.....	30
6.5 Scope for Future Work.....	30
A.1 Repository and Live Demo	34
A.2 How to Run the Project Locally.....	34
A.3 Sample Command-Line Output	34

LIST OF TABLES

Table 3.1	Key configuration parameters (config.yaml).....
Table 4.1	Technology stack used in the project.....
Table 5.1	Summary of automated test results (pytest).....
Table 5.2	Description of files generated in the outputs directory.....

LIST OF FIGURES

Fig. 3.1	System architecture of the IntelliTraffic AI pipeline.....
Fig. 3.2	Flowchart of the vehicle detection and tracking module.....
Fig. 3.3	Flowchart of the speed estimation algorithm
Fig. 3.4	Flowchart of the license plate recognition (OCR) pipeline
Fig. 3.5	Flowchart of the overspeed violation detection logic.....
Fig. 5.1	Live Processing tab – real-time progress and frame preview.....
Fig. 5.2	Overview tab – KPI cards and distribution charts
Fig. 5.3	Violations tab – flagged vehicles and offender snapshots.....
Fig. 5.4	Search tab – plate, class and speed filters.....
Fig. 5.5	Trajectories tab – heatmap and path plot.....
Fig. 5.6	Sample annotated output video frame
Fig. 5.7	Sample vehicle_records.csv output
Fig. 5.8	Application deployed on Streamlit Community Cloud

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Road traffic monitoring is one of the most visible applications of computer vision and artificial intelligence in everyday infrastructure. Manual traffic monitoring, whether by traffic police or fixed radar guns, is labour-intensive, covers only a small stretch of road at a time, and does not scale to the number of intersections and highway stretches that require monitoring in a growing city. In a country such as India, where two-wheelers, cars, buses and heavy vehicles share the same carriageway, an automated system that can simultaneously count vehicles, estimate their speed, read their license plates and flag violations offers a low-cost and scalable alternative to dedicated radar or loop-detector infrastructure.

This project, IntelliTraffic AI, was originally built as a self-contained traffic surveillance prototype and was substantially rebuilt during this summer training into a modular, tested, and research-oriented system. The system takes a traffic camera video as input and produces vehicle counts, per-vehicle speed estimates, license plate readings, an overspeed violation log, and an interactive analytics dashboard, entirely from a single camera feed and freely available open-source models.

1.2 Objectives of the Project

- To detect and continuously track vehicles (cars, motorcycles, buses and trucks) in a traffic video using a modern object detector and a multi-object tracker.
- To estimate the real-world speed of each tracked vehicle from its pixel motion, using a configurable camera calibration scheme.
- To automatically localise and read vehicle license plates using optical character recognition, with corrections tailored to the Indian plate format.
- To automatically flag vehicles that exceed a configurable speed limit, and to record supporting evidence (snapshot images and CSV logs) for each violation.

- To present all of the above through an interactive, non-technical dashboard, and to package the analysis pipeline so it can also be run headlessly from the command line.
- To build a small research toolkit around the pipeline so that its detection, tracking, speed, and OCR accuracy can be measured quantitatively rather than judged only by eye.

1.3 Scope of the Report

Chapter 2 surveys the background literature and the existing building blocks (YOLOv8, ByteTrack, EasyOCR) that this project is built upon. Chapter 3 explains the system architecture and the algorithm used in each module, supported by flowcharts. Chapter 4 describes the software implementation, technology stack, and engineering practices (configuration, testing, continuous integration) used to make the project reliable. Chapter 5 presents the results of running the system, including dashboard screenshots and sample output data. Chapter 6 concludes the report with a summary of learning outcomes, the project's relevance to current industry practice, and directions for future work.

CHAPTER 2

BASICS OF THE WORK AND LITERATURE SURVEY

2.1 Object Detection with YOLO

The “You Only Look Once” (YOLO) family of models frames object detection as a single regression problem, predicting bounding boxes and class probabilities directly from an image in one forward pass. This makes YOLO models substantially faster than earlier two-stage detectors such as Faster R-CNN, while remaining competitive in accuracy, which is why YOLO variants are the most commonly used detector in real-time traffic-monitoring literature [10]. This project uses YOLOv8 (via the Ultralytics library), restricting detections to four vehicle classes – car, motorcycle, bus and truck – so that irrelevant detections such as pedestrians or street furniture never reach the rest of the pipeline.

2.2 Multi-Object Tracking

A detector alone only tells us what is in a single frame; a tracker is needed to know that the car seen in frame 100 is the same car seen in frame 130, so that its speed and plate can be associated with one identity over time. Two tracking algorithms recur throughout the surveyed literature. DeepSORT [2] extends the earlier SORT algorithm with a learned appearance descriptor, allowing it to re-identify vehicles through short occlusions at the cost of running an additional neural network per detection. ByteTrack [1] takes a different approach: rather than discarding low-confidence detections outright, it associates almost every detection box – including low-score ones caused by partial occlusion – with existing tracks using motion cues alone, which keeps it lightweight while reducing fragmented tracks. This project uses ByteTrack as its primary tracker (with BoT-SORT available as a configurable alternative) because it avoids the overhead of a separate appearance-embedding network while still reporting stable IDF1 scores on standard benchmarks.

2.3 Vision-Based Vehicle Speed Estimation

Estimating a vehicle's real-world speed from a single (monocular) camera is fundamentally a problem of converting pixel displacement into metric distance, which requires knowing how the camera's image plane relates to the road plane. A widely cited approach pairs a YOLO detector with a formal camera calibration procedure and DeepSORT tracking to obtain two-dimensional vehicle positions before computing speed [3]. More recent work has explored calibration-free alternatives: one recent method estimates speed using a YOLO-based vehicle keypoint detector and a metric vehicle template combined with a homography, entirely avoiding the need to measure the camera's calibration parameters in advance [4]. Onboard (moving-camera) variants of the same problem have also been studied, for instance by combining YOLOv8 detection with distance priors and kernel regression smoothing [5]. This project adopts a middle ground: it supports both a simple metres-per-pixel calibration and a four-point homography calibration, letting the user trade off setup effort against accuracy depending on the camera angle available.

2.4 Automatic License Plate Recognition (ALPR)

ALPR systems typically split into two stages: locating the plate region within a vehicle image, and reading the characters inside it. Classical localisation methods rely on edge density and the characteristic aspect ratio of a plate, while character reading is usually delegated to an OCR engine. A recent systematic comparison of OCR engines for ALPR – Tesseract, EasyOCR, PaddleOCR and TrOCR – found that engine choice significantly affects accuracy under difficult lighting and script conditions [7]. Several applied studies pair YOLO detectors directly with EasyOCR for plate reading; one such study evaluating YOLO-based Indonesian license plate detection paired with EasyOCR is structurally very close to the approach used in this project [8], while other detailed studies of deep-learning-based plate detection and recognition give useful accuracy baselines to compare against [6]. Indian license plates introduce their own formatting quirks (a two-letter state code, a numeric RTO code, an optional series of letters, and a four-digit number), so this project adds a positional character-confusion correction step (correcting common OCR confusions such as O/0, I/1 and S/5 based on where they occur in the plate string) on top of a general-purpose EasyOCR reader [9].

2.5 Traffic Analytics Dashboards

A detection-and-tracking pipeline only becomes useful to a traffic operator once its output is presented in an interpretable form. Streamlit [11] was chosen for this project's dashboard because it allows a data pipeline written in Python to be exposed as an interactive web application with minimal additional front-end code, and because it can be deployed free of charge on Streamlit Community Cloud, making the finished system directly usable by anyone with the repository link rather than requiring a local Python installation.

2.6 Summary of the Literature Survey

Across the literature surveyed, three recurring design choices stood out: the pairing of a YOLO detector with either DeepSORT or ByteTrack for tracking, the use of either a simple pixel-to-metre scale or a full homography for speed calibration, and the pairing of a plate detector with a general-purpose OCR engine such as EasyOCR or PaddleOCR. IntelliTraffic AI deliberately follows the lighter-weight branch of each choice (YOLOv8 + ByteTrack, dual calibration modes, and EasyOCR with a regional post-processing step) so that the complete pipeline can run in real time on commodity hardware and be hosted on a free-tier cloud service, while still remaining close enough to the reviewed literature that its outputs can be benchmarked against published numbers.

CHAPTER 3

SYSTEM DESIGN AND METHODOLOGY

3.1 Overall System Architecture

Figure 3.1 shows the end-to-end architecture of IntelliTraffic AI. A video is read frame by frame; each frame passes through YOLOv8 detection restricted to vehicle classes, then through ByteTrack for identity association. From each tracked vehicle, two independent branches run in parallel: a speed estimator that converts the vehicle's pixel motion into a smoothed km/h value, and a plate-reading branch that periodically crops the vehicle and runs plate localisation plus OCR. Both branches feed a violation detector, which flags a vehicle only after it has been observed above the configured speed limit for several consecutive frames. All of this information is drawn back onto the video frame, written out as a browser-playable H.264 video, and logged to a set of CSV files; the Streamlit dashboard then reads these outputs to present the interactive analytics views described in Chapter 5.

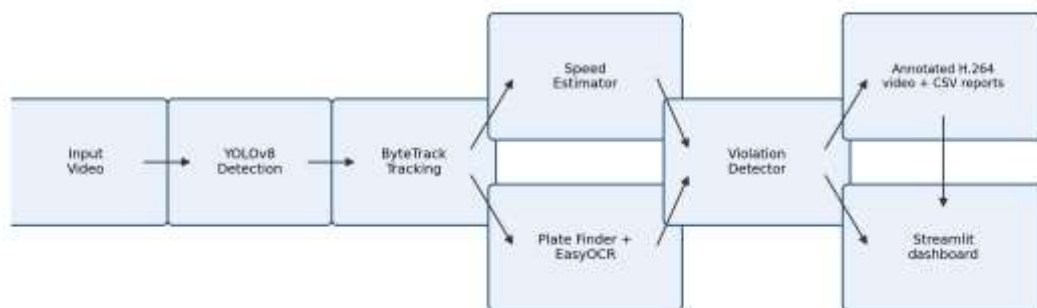


Fig. 3.1: System architecture of the IntelliTraffic AI pipeline

3.1.1 Configuration-Driven Design

Every tunable value in the pipeline – confidence and IoU thresholds, the set of tracked classes, the calibration mode and its parameters, the OCR frequency, and the speed limit – is defined centrally in a single configuration file (`config.yaml`) and can be overridden per run, either from the dashboard sidebar or from the command line. Table 3.1 lists the most important parameters.

Table 3.1: Key configuration parameters (`config.yaml`)

Parameter	Default	Purpose
model.conf	0.35	Minimum detection confidence for YOLOv8
model.classes	car, motorcycle, bus, truck	Vehicle classes retained after detection
model.tracker	bytetrack.yaml	Multi-object tracker configuration used
speed.mode	simple	Calibration mode: simple (m/px) or homography
speed.limit_kmh	60	Speed limit above which a vehicle may be flagged
speed.consecutive_frames	5	Frames above the limit required before flagging
ocr.every_n_frames	10	OCR read frequency per tracked vehicle

3.2 Vehicle Detection and Tracking Algorithm

Figure 3.2 shows the per-frame detection and tracking flow. Every frame is passed to YOLOv8; each resulting detection is checked against the configured vehicle class list before being handed to ByteTrack, so that non-vehicle detections never acquire a track ID. ByteTrack either extends an existing track or starts a new one, and the resulting track history (class votes, centroid path, and frame count) is updated for use by the later stages.

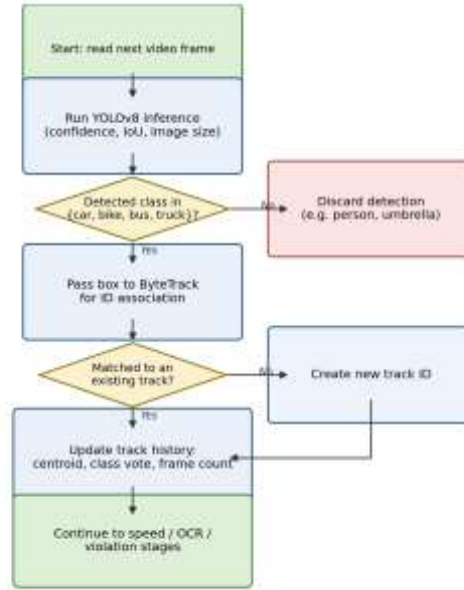


Fig. 3.2: Flowchart of the vehicle detection and tracking module

3.3 Speed Estimation Algorithm

Figure 3.3 shows how a raw pixel position is converted into a smoothed speed value. The bottom-centre point of each vehicle's bounding box (a reasonable proxy for where the vehicle contacts the road) is mapped from image pixels into real-world metres, either through a fixed metres-per-pixel scale or through a perspective (homography) transform computed from four user-supplied point correspondences. Successive world-coordinate positions are held in a short sliding window per vehicle; once enough samples have accumulated, the distance between the earliest and latest sample in the window is divided by the elapsed time and converted to km/h. Finally, an exponential moving average is applied across frames to suppress the jitter that would otherwise arise from small detection or tracking noise.

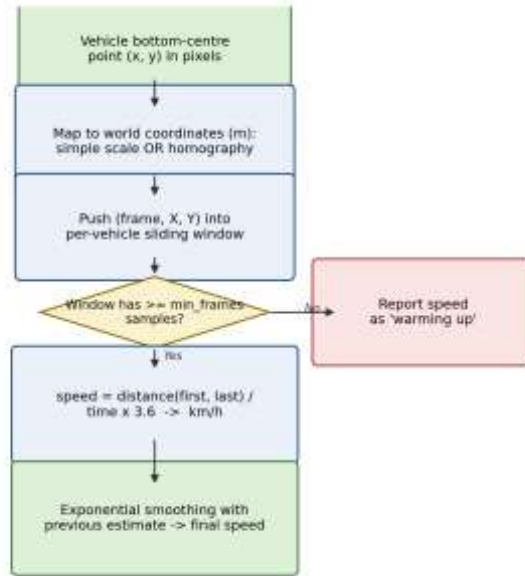


Fig. 3.3: Flowchart of the speed estimation algorithm

3.4 License Plate Recognition Algorithm

Figure 3.4 shows the plate-reading pipeline, which runs periodically (rather than on every frame) for each tracked vehicle to keep the system real-time. The vehicle crop is first searched for a plate-shaped region using edge detection and a contour aspect-ratio filter, falling back to the lower portion of the crop if no clear candidate is found. The candidate region is upscaled and contrast-enhanced before being passed to EasyOCR. The raw OCR string is cleaned, and a positional character-confusion correction is applied using the known structure of an Indian plate. Because a single frame's reading can be wrong, every reading for a given vehicle ID is accumulated into a confidence-weighted vote, and the highest-scoring candidate string is reported as that vehicle's plate.

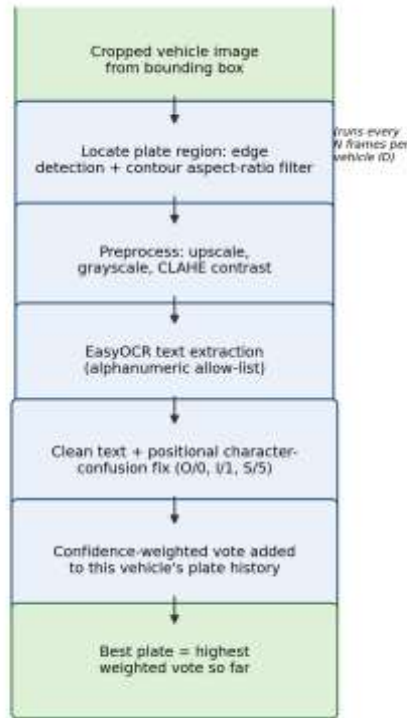


Fig. 3.4: Flowchart of the license plate recognition (OCR) pipeline

3.5 Overspeed Violation Detection Logic

Figure 3.5 shows the violation logic. A vehicle is not flagged the instant its speed crosses the configured limit; instead, a per-vehicle counter tracks how many consecutive frames the vehicle has spent above the limit, and only once this counter reaches a configurable threshold is the vehicle flagged, at which point a snapshot image is saved and the event is written to the violations log. This consecutive-frame requirement was a deliberate design decision to reduce false positives caused by a single noisy speed reading.

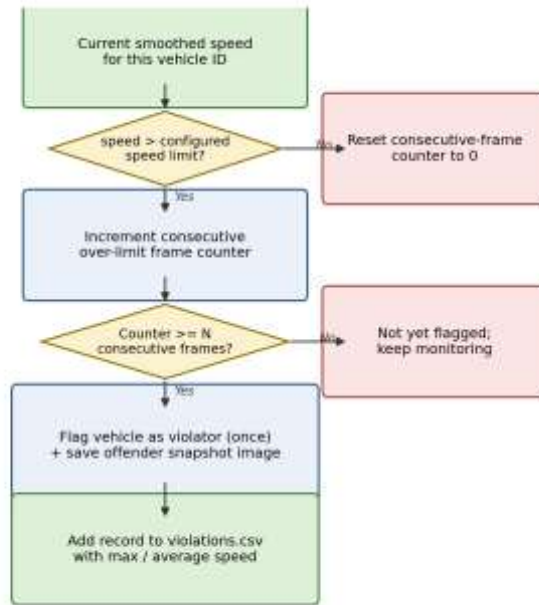


Fig. 3.5: Flowchart of the overspeed violation detection logic

CHAPTER 4

IMPLEMENTATION DETAILS

4.1 Technology Stack

Table 4.1 lists the principal libraries and tools used to implement the project.

Table 4.1: Technology stack used in the project

Layer	Technology	Role in the project
Detection	Ultralytics YOLOv8 (PyTorch)	Vehicle detection restricted to 4 classes
Tracking	ByteTrack / BoT-SORT	Multi-object tracking, per-vehicle identity
Computer vision	OpenCV	Video I/O, drawing, classical plate localisation
OCR	EasyOCR	License plate character recognition
Data handling	pandas, NumPy	Records, analytics, CSV I/O
Dashboard	Streamlit, Plotly	Interactive web dashboard and charts
Video encoding	ffmpeg (imageio-ffmpeg)	Browser-playable H.264 re-encoding
Testing	pytest	Unit and pipeline smoke tests
CI/CD	GitHub Actions	Automated linting and testing on every push
Deployment	Streamlit Community Cloud	Free hosting of the live dashboard

4.2 Project Structure

The codebase is organised into an app/ package containing one module per pipeline responsibility (detection, tracking, speed, ocr, violations, trajectory, analytics, pipeline, utils, config), a top-level app.py for the Streamlit dashboard and main.py for the command-line interface, a research/ package containing the evaluation and benchmarking scripts described in

Section 4.5, and a tests/ package containing the automated test suite. This separation means each module can be tested and reasoned about independently, and the same pipeline code is shared by the dashboard, the CLI, and the research scripts.

4.3 Key Implementation Choices

4.3.1 Strict Vehicle-Class Filtering

Although YOLOv8's default weights are trained on the full 80-class COCO dataset, this project applies a defence-in-depth filter: the detector is asked to restrict itself to the four vehicle-class IDs at inference time, and the resulting detections are filtered again in Python against the configured class list. This double filtering ensures that a misconfiguration in one layer cannot let a non-vehicle object (for example, a pedestrian or a street umbrella) leak through and receive a track ID.

4.3.2 Browser-Playable Video Output

OpenCV's default video writer produces MP4 files encoded in a codec that many browsers, including the one embedded in the Streamlit dashboard, cannot play directly. To resolve this, every processed video is re-encoded using ffmpeg to H.264 with yuv420p pixel format and the +faststart flag, with a fallback to the raw file if ffmpeg is unavailable, so the dashboard's video player is guaranteed a compatible file whenever possible.

4.3.3 Graceful Failure Handling

Every stage of the pipeline is written to degrade gracefully rather than crash: an unreadable or corrupt video raises a clear, user-facing error instead of an unhandled exception; a vehicle crop too small for reliable OCR is simply skipped; and an empty or missing CSV file is treated as “zero vehicles” by the analytics module rather than causing the dashboard to fail to load.

4.4 Testing and Quality Assurance

A pytest suite covering every module was written alongside the implementation, using a synthetic video with a known ground-truth motion so that the reported speed could be checked against a hand-calculated expected value rather than an assumption. Table 5.1 in the next chapter summarises the results of running this suite. A GitHub Actions workflow runs the linter

and the full test suite automatically on every push, so that a change which breaks an existing feature is caught immediately rather than during a manual demonstration.

4.5 Research and Evaluation Toolkit

In addition to the production pipeline, a small research/ package was implemented to allow the system's accuracy to be measured quantitatively rather than judged only by eye. It includes scripts to compute detection precision/recall/mAP against a labelled dataset, tracking quality (MOTA, MOTP, IDF1, ID-switch count) against ground-truth trajectories, speed accuracy (MAE, RMSE, MAPE and a Bland–Altman analysis) against known vehicle speeds, plate-reading accuracy (exact-match rate and character error rate), an ablation sweep across model size and image resolution, and a latency/throughput benchmark comparing CPU and GPU execution. These scripts are described further, with the exact commands and required ground-truth formats, in the project's research/README.md file.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Automated Test Results

Before evaluating the system visually, the full automated test suite was executed to confirm that every module behaves as designed. Table 5.1 summarises the outcome.

Table 5.1: Summary of automated test results (pytest)

Test module	What it verifies	Result
test_detection.py	Non-vehicle classes are correctly filtered out	Passed
test_speed.py	Speed maths for both calibration modes	Passed
test_ocr.py	Plate text cleanup, confusion-correction, voting	Passed
test_violations.py	Consecutive-frame violation logic	Passed
test_analytics.py	KPI, filter and empty-CSV handling	Passed
test_pipeline.py	Full pipeline smoke test on synthetic video	Passed
test_research_metrics.py	Error-metric and text-distance functions	Passed

On a synthetic test video with a rectangle moving at a known, constant pixel speed, the pipeline's reported speed matched the hand-calculated expected value, and the resulting video, CSV files and metadata file were all produced and correctly formatted, confirming that the pipeline's core arithmetic and file-handling are correct independent of detector accuracy.

5.2 Dashboard Walkthrough

The following screenshots demonstrate the working dashboard on an actual traffic video.

5.2.1 Live Processing

The Live Processing tab shows a progress bar, an estimated time remaining, and a live preview of the frame currently being processed while the video is analysed.



Fig. 5.1: Live Processing tab – real-time progress and frame preview

5.2.2 Overview

Once processing completes, the Overview tab summarises the run with KPI cards (total vehicles, average speed, maximum speed, violation count) alongside a vehicle-type distribution chart and a speed-distribution histogram.

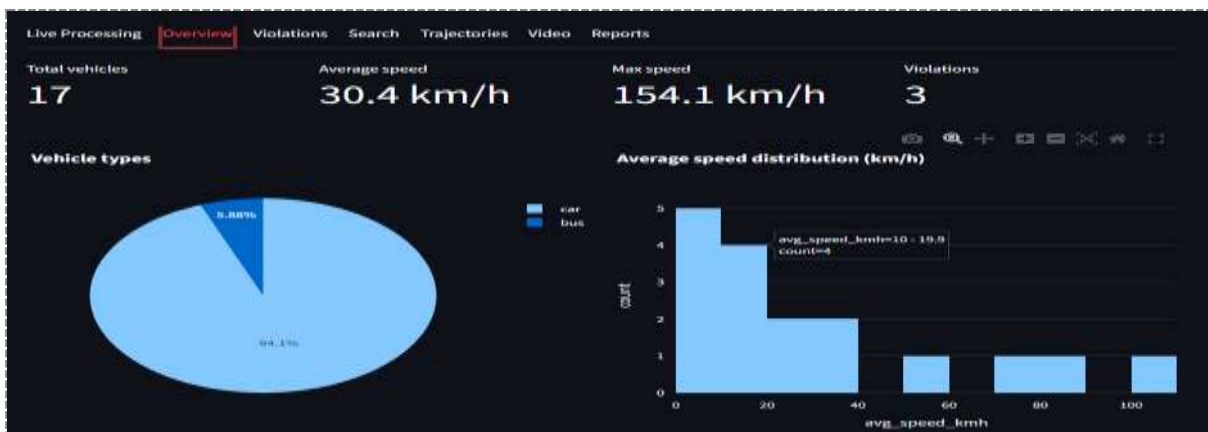


Fig. 5.2: Overview tab – KPI cards and distribution charts

5.2.3 Violations

The Violations tab lists every vehicle that was flagged for exceeding the configured speed limit, along with its offender snapshot image.

Live Processing Overview Violations Search Trajectories Video Reports										
	vehicle_id	class	plate	first_frame	last_frame	duration_s	avg_speed_kmh	max_speed_kmh	overspeed	
	4	11	car	RINGLTD	81	181	4.04	50.42	102.75	☐
	12	37	car		139	201	2.52	75.71	115.66	☐
	15	48	car		174	201	1.12	105.8	154.11	☐

#11 · 103 km/h · RINGLTD #37 · 116 km/h · no plate #48 · 154 km/h · no plate

Fig. 5.3: Violations tab – flagged vehicles and offender snapshots

5.2.4 Search

The Search tab allows partial, case-insensitive plate lookup alongside vehicle-class and speed-range filters.

Live Processing

Overview

Violations

Search

Trajectories

Videos

Reports

Search plate number (partial, case-insensitive)

Vehicle type

Choose options

Max speed range (km/h)

0.00

154.11

☐ Only violations

17 vehicle(s)

	vehicle_id	class	plate	first_frame	last_frame	duration_s	avg_speed_kmh	max_speed_kmh	overspeed
0	1	car		5	28	1.16	12.48	18.2	☐
1	3	car		13	36	0.96	14.85	22.3	☐
2	7	car		47	172	5.04	4.58	11.03	☐
3	8	car	43579	58	201	5.76	5.17	16.83	☐
4	11	car	RINGLTD	81	181	4.04	50.42	102.75	☒
5	17	bus	4931	93	201	4.4	6	12.71	☐
6	18	car		97	149	2.12	2.12	9.32	☐

Fig. 5.4: Search tab – plate, class and speed filters

5.2.5 Trajectories

The Trajectories tab overlays a density heatmap of all vehicle paths on a background frame, alongside a per-vehicle path plot.



Fig. 5.5: Trajectories tab – heatmap and path plot

5.3 Sample Output Artefacts

Figure 5.6 shows a representative frame from the annotated output video, with bounding boxes, vehicle IDs, estimated speed, and recognised plate text drawn directly on the frame, and a red bounding box used for any vehicle that has been flagged as a violator.



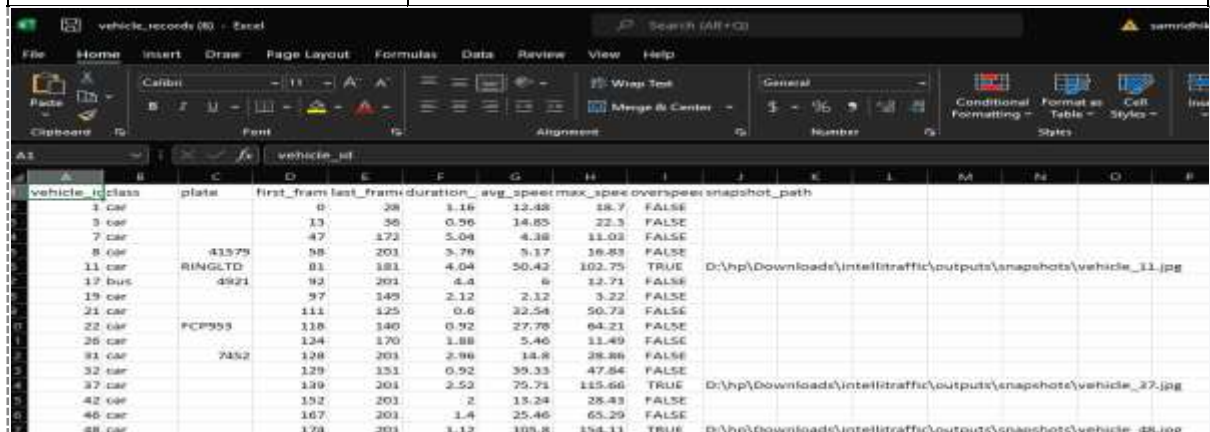
Fig. 5.6: Sample annotated output video frame

Table 5.2 summarises the files written to the outputs directory after a run, and Figure 5.7 shows a sample of vehicle_records.csv opened in a spreadsheet application.

Table 5.2: Description of files generated in the outputs directory

File	Contents
final_output.mp4	Annotated, browser-playable H.264 video
vehicle_records.csv	One row per vehicle: class, plate, timing, speed, violation flag

File	Contents
violations.csv	Subset of vehicle_records.csv containing only flagged vehicles
trajectories.csv	Frame-by-frame (x, y) position for every tracked vehicle
speed_timeline.csv	Per-frame speed samples used to draw the speed-over-time chart
trajectory_heatmap.png	Accumulated vehicle-density heatmap image
run_metadata.json	Configuration used, device, timing and library versions for the run



vehicle_id	class	plate	first_frame	last_frame	duration	avg_speed	max_speed	overspeed	snapshot_path
1	car		0	28	1.16	12.48	18.7	FALSE	
3	car		13	36	0.96	14.85	22.3	FALSE	
7	car		47	173	5.04	4.38	11.03	FALSE	
8	car	43579	98	201	3.76	5.17	16.83	FALSE	
11	car	RINGLTD	81	181	4.04	50.42	102.75	TRUE	D:\hp\Downloads\IntelliTraffic\outputs\snapshots\vehicle_11.jpg
17	bus	4921	93	201	4.4	6	12.71	FALSE	
19	car		97	149	2.12	2.12	3.22	FALSE	
21	car		111	125	0.6	32.54	50.73	FALSE	
22	car	PCP959	118	140	0.92	27.78	64.21	FALSE	
26	car		124	170	1.88	5.40	11.49	FALSE	
31	car	7452	128	201	2.96	14.8	38.88	FALSE	
32	car		129	151	0.92	39.33	47.64	FALSE	
37	car		139	201	2.52	75.71	115.66	TRUE	D:\hp\Downloads\IntelliTraffic\outputs\snapshots\vehicle_37.jpg
42	car		152	201	2	15.24	28.43	FALSE	
46	car		167	201	1.4	25.46	65.29	FALSE	
58	car		178	201	1.12	38.8	105.1	TRUE	D:\hp\Downloads\IntelliTraffic\outputs\snapshots\vehicle_58.jpg

Fig. 5.7: Sample vehicle_records.csv output

5.4 Deployment

The dashboard was deployed to Streamlit Community Cloud so that it can be accessed from any browser without a local installation. Figure 5.8 shows the deployed application running live.

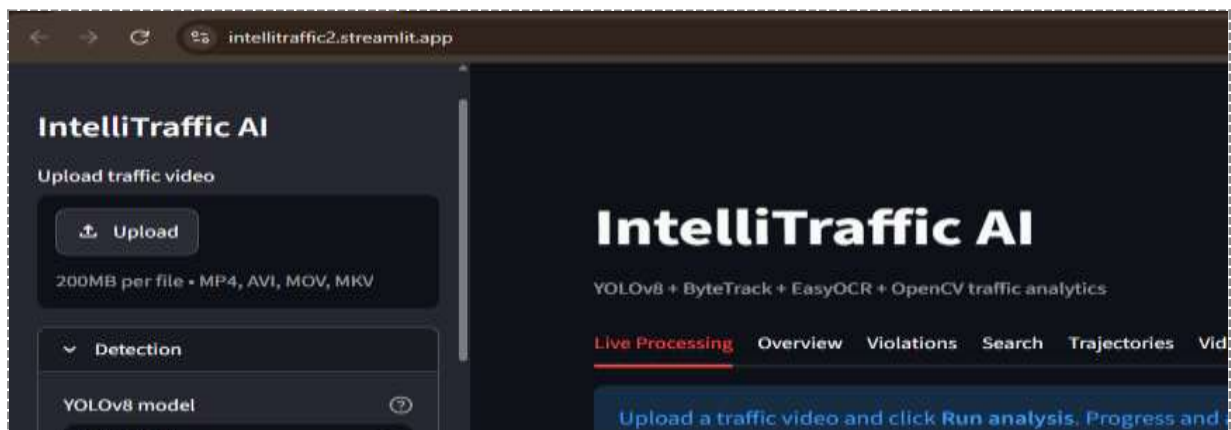


Fig. 5.8: Application deployed on Streamlit Community Cloud

5.5 Discussion

The results confirm that the rebuilt pipeline meets its stated objectives: vehicles are detected and tracked with non-vehicle classes correctly excluded, speed is computed and smoothed consistently with the underlying arithmetic, plates are read and cleaned using an Indian-plate-aware correction step, and violations are logged only after a sustained overspeed event rather than a single noisy frame. The most important caveat, discussed further in Chapter 6, is that the accuracy of the speed values reported by the dashboard depends entirely on how carefully the camera has been calibrated for the specific scene being analysed; the pipeline's internal logic was verified against a known ground truth in Section 5.1, but real-world accuracy on an uncalibrated camera should not be assumed without a dedicated calibration step.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Summary of Work Carried Out

This summer training project took an existing single-file traffic-surveillance prototype and rebuilt it into a modular, configuration-driven, and automatically tested system covering vehicle detection, multi-object tracking, dual-mode speed estimation, Indian-plate-aware license plate recognition, consecutive-frame overspeed violation detection, trajectory visualisation, and an interactive Streamlit dashboard, in addition to a command-line interface and a dedicated research toolkit for quantitative evaluation. Every module was covered by an automated test, and a continuous-integration workflow was set up so that future changes cannot silently break an existing feature.

6.2 Learning Outcomes

- Practical experience combining three separate deep-learning components (an object detector, a multi-object tracker and an OCR engine) into a single, coherent real-time pipeline.
- An understanding of the geometry underlying monocular vehicle speed estimation, including the trade-off between a simple metres-per-pixel scale and a full homography calibration.
- Hands-on experience writing automated tests for a computer-vision pipeline using a synthetic, ground-truth-controlled video, rather than relying solely on visual inspection.
- Exposure to reading and critically comparing recent research papers on vehicle speed estimation and automatic license plate recognition, and translating that comparison into concrete design decisions (choice of tracker, choice of OCR engine, and a regional post-processing step for plate text).
- Experience packaging and deploying a Python computer-vision application as a public, cloud-hosted web dashboard using Streamlit Community Cloud.

6.3 Relevance to Current Industry Practice

Intelligent Transportation Systems (ITS) are an active area of investment for both government road-safety agencies and private mobility companies, and camera-based analytics of exactly the kind built in this project – vehicle counting, speed enforcement, and automatic number-plate recognition – are already deployed at scale on Indian highways and toll plazas. The skills exercised in this project (combining detection, tracking and OCR models; calibrating a camera for metric measurement; and building a tested, deployable analytics application) map directly onto the day-to-day work of computer-vision and ITS engineering teams in this space.

6.4 Limitations

- Speed accuracy is only as good as the camera calibration; no automatic calibration is currently performed.
- A single camera cannot resolve true depth, so the road-plane assumption breaks down for elevated or heavily angled camera placements.
- OCR accuracy degrades in low light, at night, or when a plate is captured at a steep angle, since the current plate localiser uses classical edge-based detection rather than a learned plate detector.
- Ground-truth data for the research/ evaluation scripts (labelled vehicles, true speeds, true plate text) must be collected separately; none is bundled with the project.

6.5 Scope for Future Work

- Lane detection and lane-change analysis, to support per-lane speed and violation statistics.
- A learned license-plate detector to replace the current classical localisation step, particularly to improve night-time and oblique-angle performance.
- Accident detection through sudden trajectory or speed discontinuities.
- Native support for live RTSP camera streams rather than only pre-recorded video files.
- Multi-camera fusion, so that a single vehicle can be tracked across the fields of view of several cameras along a corridor.

- Cloud database integration, so that violation records persist beyond a single session and can be queried across multiple monitoring sites.

REFERENCES

- [1] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in Proc. European Conf. on Computer Vision (ECCV), 2022. [Online]. Available: arXiv:2110.06864.
- [2] N. Wojke, A. Bewley, and D. Paulus, “Simple Online and Realtime Tracking with a Deep Association Metric,” in Proc. IEEE Int. Conf. on Image Processing (ICIP), 2017, pp. 3645–3649.
- [3] “Vehicle speed measurement method using monocular cameras,” Scientific Reports, 2025. [Online]. Available: doi:10.1038/s41598-025-87077-6.
- [4] “Calibration-Free Vehicle Speed Estimation: A Monocular Keypoint-Template Approach,” arXiv:2608.16785, 2026.
- [5] M. López-Pola et al., “Estimating the Speed of Nearby Vehicles with a Single Onboard Camera by Smooth Kernel Regression,” Int. J. Neural Systems, vol. 36, no. 4, 2026.
- [6] R. Laroca, E. Severo, L. A. Zanlorensi, and L. S. Oliveira et al., “Efficient and robust vehicle license plate detection and recognition using deep learning,” Image and Vision Computing, vol. 104, p. 104004, 2021.
- [7] “OCR Engines for License Plate Recognition: A Comparative Study of Tesseract, EasyOCR, PaddleOCR, and TrOCR,” Journal of Applied Informatics and Computing, vol. 10, no. 4, 2026.
- [8] R. N. Suharto et al., “Comparative Evaluation of YOLO-based Models for Indonesian License Plate Detection and Recognition with EasyOCR,” Procedia Computer Science, vol. 269, pp. 943–952, 2025.
- [9] Jaided AI, “EasyOCR: Ready-to-use OCR with 80+ Languages Supported,” GitHub repository, 2020. [Online]. Available: <https://github.com/JaidedAI/EasyOCR>.
- [10] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” GitHub repository, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>.

[11] Streamlit Inc., “Streamlit Documentation.” [Online]. Available: <https://docs.streamlit.io>. [Accessed: Sep. 2026].

APPENDICES

A.1 Repository and Live Demo

Source code: https://github.com/sam-coolshrestha/Intellitrffic_2.0

Live dashboard: <https://intellitrffic2.streamlit.app/>

A.2 How to Run the Project Locally

1. Clone the repository and create a virtual environment.
2. Install dependencies: `pip install -r requirements.txt`
3. Launch the dashboard: `streamlit run app.py`
4. Or run headlessly: `python main.py --input <video path> --speed-limit 60`

A.3 Sample Command-Line Output

```
PS D:\hp\Downloads\intellitrffic> python -m venv .venv
PS D:\hp\Downloads\intellitrffic> .venv\Scripts\Activate.ps1
(.venv) PS D:\hp\Downloads\intellitrffic> streamlit run app.py
2026-09-29 13:23:29.600 Uvicorn server started on :::8501
```

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8501>